

SDK Hands-on Workshop

Nordic / u-blox / Avnet

Chapter 1: IDE Introduction & Blinky

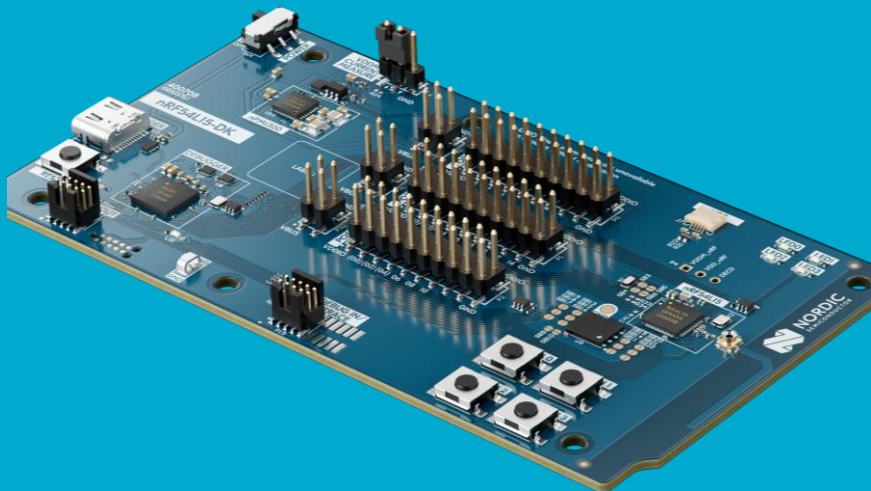
June 2026

nRF54L15-DK / EVK-NORA-B2

nRF Connect SDK v3.3.0



NORDIC[®]
SEMICONDUCTOR



Hands-on

Working with the Blinky sample, the Hello World for Embedded

Preparations

- IDE, SDK and Toolchain Installation
 - SEGGER J-Link driver
 - Visual Studio Code
 - nRF Connect Extension Pack for Visual Studio Code
 - nRF Connect SDK + Toolchain

Step-by-step guide:

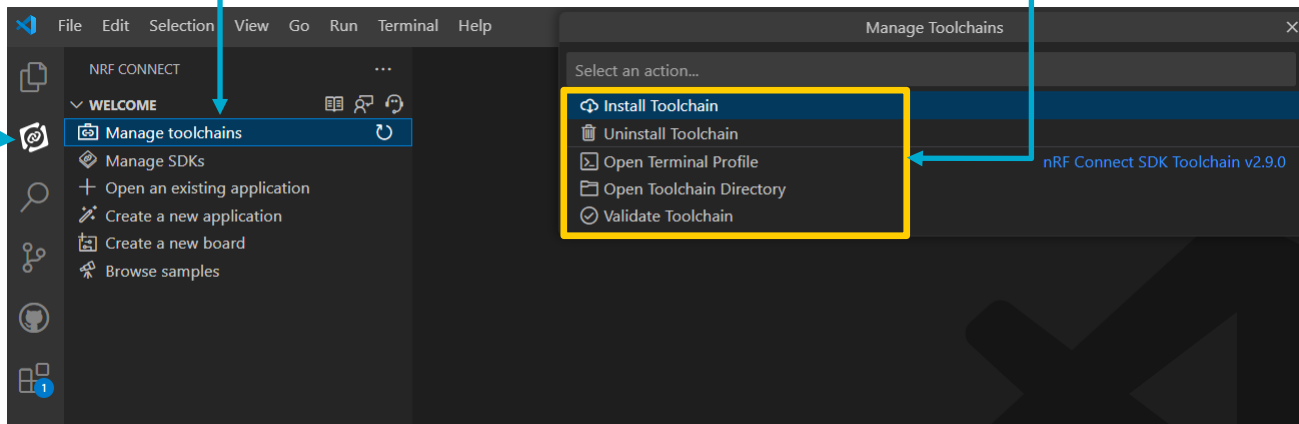
<https://academy.nordicsemi.com/topic/exercise-1-1/>

Manage Toolchain

Select the nRF Connect Extension

Manage toolchains

Install Toolchain, default installation path
<drive>:\ncs\toolchain\<build version>

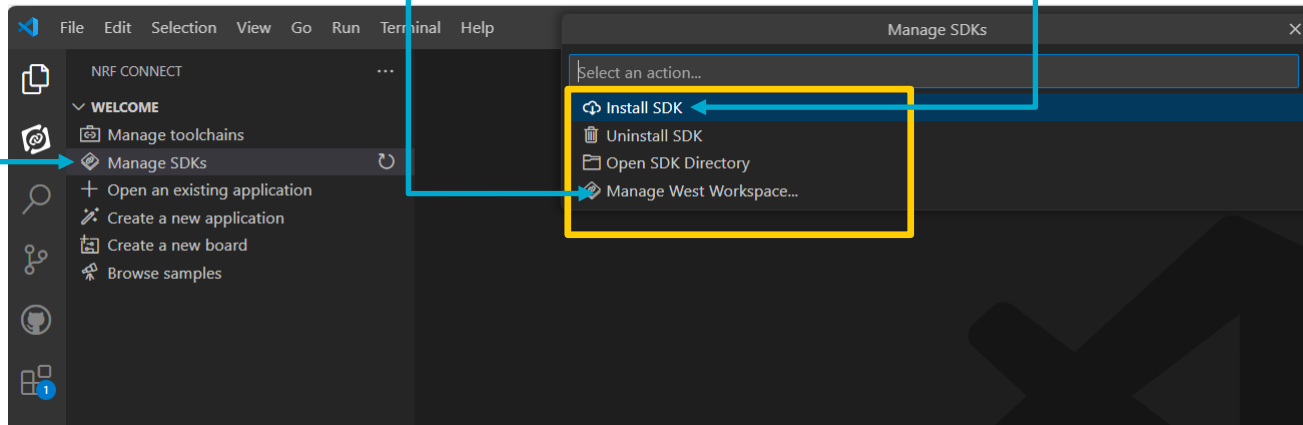


Manage SDK's installations

Manage SDKs

Create/open
Workspace projects

Install SDK, default installation path
<drive>:\ncs\v3.3.0



Agenda

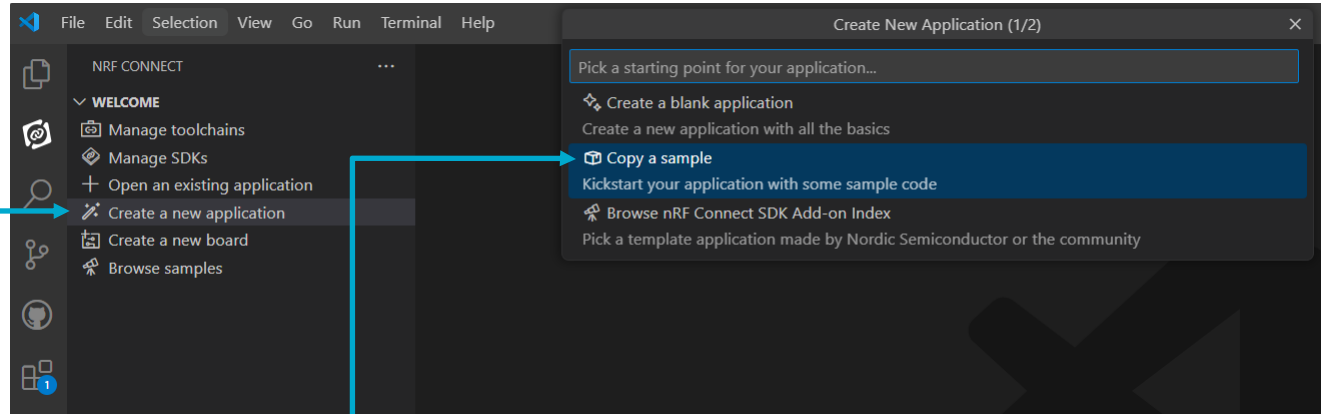
Chapter 1

- Navigating the IDE
- Adding a custom board (UBX EVK NORA B2)
- Building the Zephyr “blinky” sample
- Understanding the source code
- Modifying the devicetree using different approaches
- (opt.) Create custom kernel configuration (Kconfig) settings
- (opt.) Change while loop into a Zephyr WorkQueue thread

Navigating the IDE

Building a application

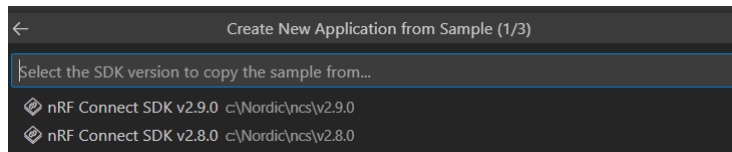
Create a new application



Copy a sample

This will create a copy from a sample or application out of the SDK repository. This will create a stand-alone application.

Select the version of the SDK

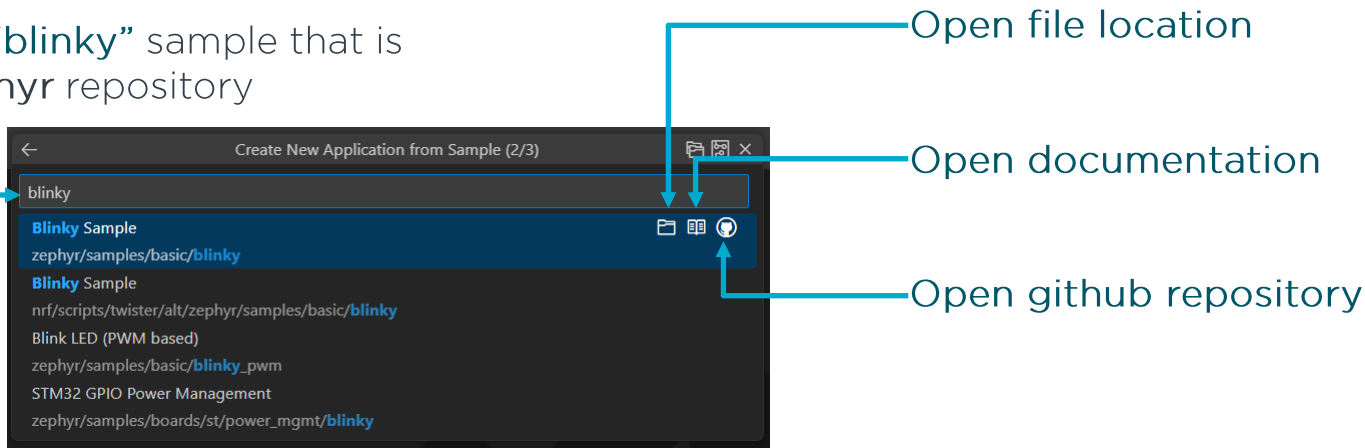


Copy a sample

This will ask you to select a SDK version to browse and copy the samples or application.

Selecting a sample: blinky

Search for the “**blinky**” sample that is part of the **zephyr** repository



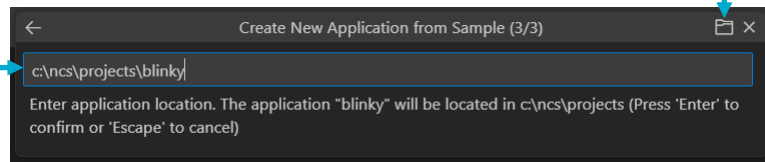
If the location of the sample/application starts with “**zephyr**” this sample is provided by Zephyr.

If the location starts with “**nrf**” the sample is provided by Nordic Semiconductor and part nRF Connect SDK

Location of new application

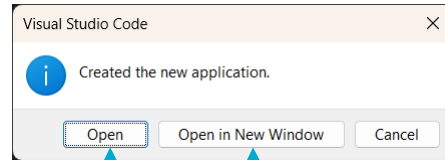
Full path to location of project.
By default the last used location
followed by the **application name**.

Use icon to browse for target file
location of new application



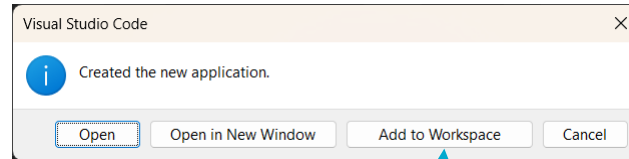
Where to open the new application

Open project in current
Visual Code instance



Open a new
Visual Code instance and
open the project

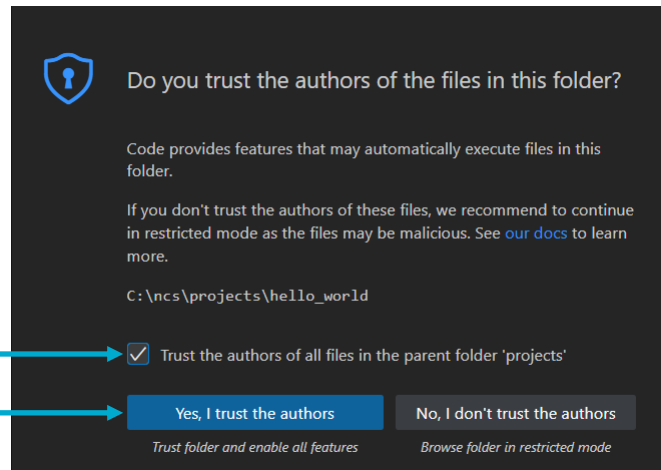
Add project to workspace in
current Visual Code instance



Permission to access project location

Optional to trust the complete parent folder of the project

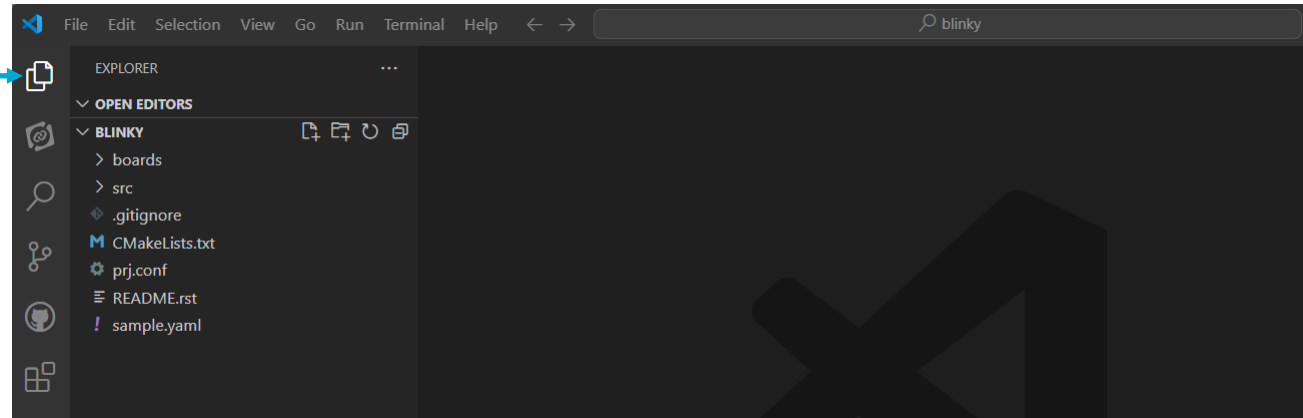
Mandatory for Visual Code to trust the location of the project



Application created

The new application based on the sample has been created in a Visual Code Workspace (not to confuse with West Workspace)

VC has opened the project in the Explorer view



Adding custom boards

Example: Adding UBX EVK-NORA-B2

Code snippets

<https://github.com/kkotinkar/SDK-Workshop>



Adding custom board files 1/3

- The build chain allows integrating custom board files that are not part of your SDK/Zephyr
 - These can be your own, custom board files, or board files from different vendors
 - A clean method is to keep the SDK installation directories untouched
 - We will utilize the CMake Variable *BOARD_ROOT* and set this up within the IDE

1. Download and extract your board files

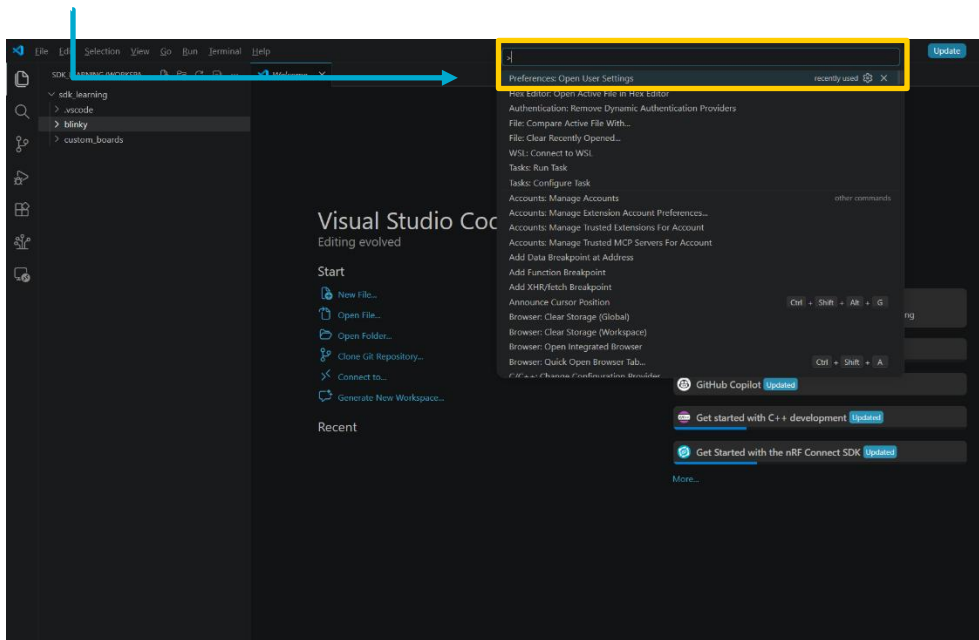
- Download the provided `ubx_evknorab2_zephyr_main.zip`
- Unzip and copy the files into a directory of your choice
- The custom boards folder needs to contain the *boards* folder

```
C:\...\custom_boards\  
└── boards\  
    └── u-blox\  
        └── ubx_evknorab2\
```

Adding custom board files 2/3

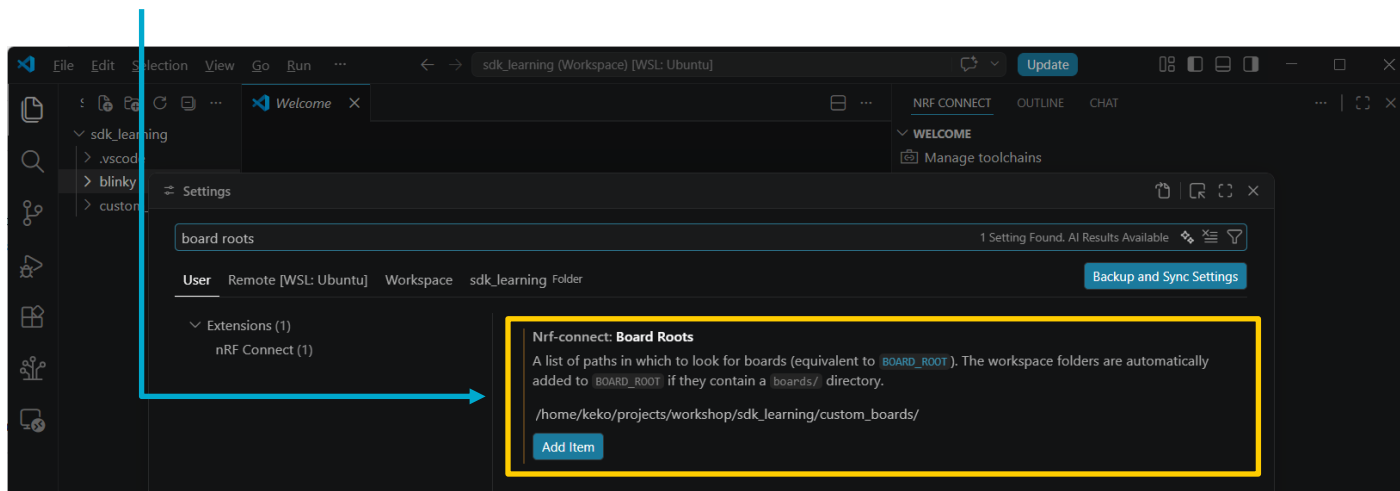
2. Open the VS Code user settings

- Use the top command bar and enter: **> Preferences Open User Settings** (including ">")



Adding custom board files 3/3

3. Search for “board roots” and add the path to your custom boards folder
 - This is a setting of the nRF Connect Extension
 - You can choose whether to apply this globally, only in your workspace or per project



Build configurations

Setting up build configurations 1/3

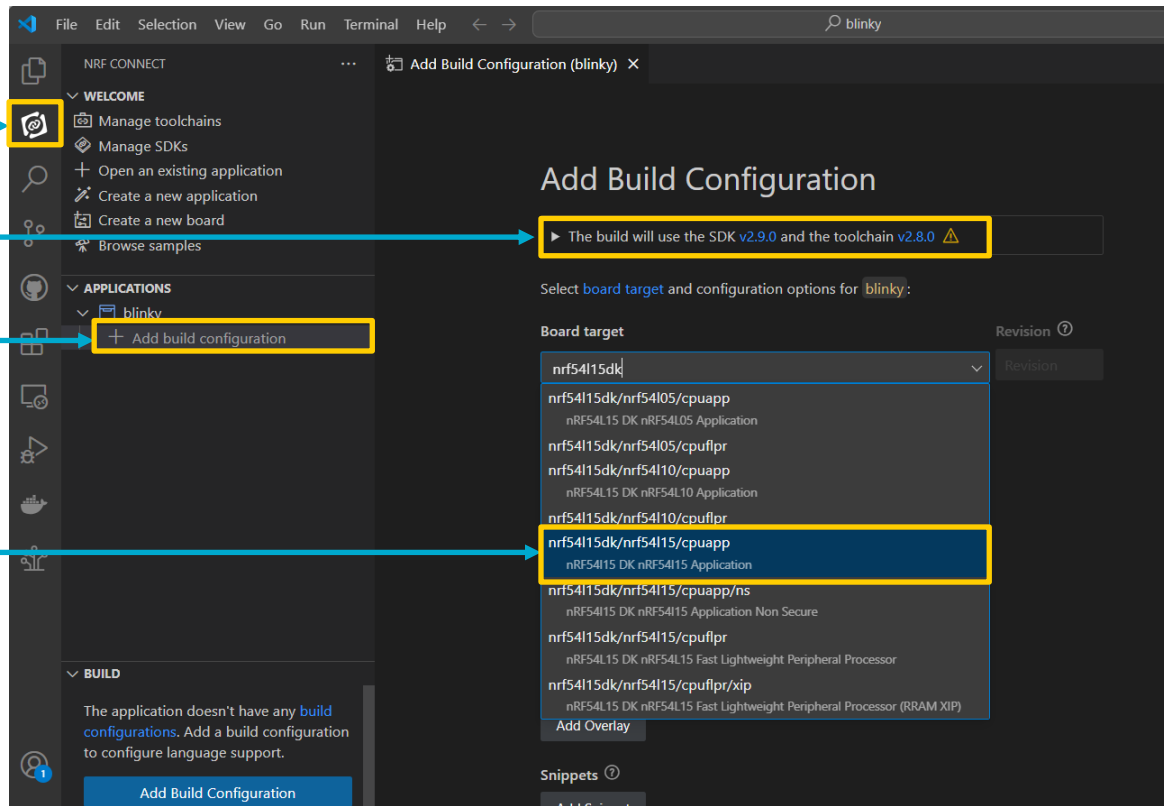
Select the nRF Connect Extension

Select the version of the SDK and toolchain

Add build configuration

Select the target board
nrf54l15dk/nrf54l15/cpuapp OR
ubx_evknorab2/nrf54l15/cpuapp

- Board nRF54L15DK / EVK-NORA-B2
- SoC nRF54L15
- Core cpuapp



Select Kconfig configuration if empty the **default** is used

Select device tree overlay if empty the **default** is used

Add code snippet common build code

Give a name for the current build directory

Use Build system default or Use sysbuild

The screenshot shows the 'SDK' configuration window for nRF Connect SDK v3.0.0. The following sections are highlighted with yellow boxes:

- Base configuration files (Kconfig fragments):** Includes a 'Select...' dropdown and a 'Browse' button.
- Extra Kconfig fragments:** Includes a 'Select...' dropdown and a 'Browse' button.
- Base Devicetree overlays:** Includes a 'Select...' dropdown and a 'Browse' button.
- Extra Devicetree overlays:** Includes a 'Select...' dropdown and a 'Browse' button.
- Snippets:** Includes a 'Select...' dropdown.
- Optimization level (size, speed, or debugging):** Includes a dropdown menu with 'Optimize for debugging (-Og)' selected and a note 'Also adds additional thread information'.
- Extra CMake arguments:** Includes an 'Add Argument' button.
- Build directory name:** Includes a text field with 'build' and a 'Browse' button.
- System build (sysbuild):** Includes radio buttons for 'Build system default' (selected), 'Use sysbuild', and 'No sysbuild'.

Blue arrows point from external text blocks to these highlighted sections:

- From 'Select Kconfig configuration...' to 'Base configuration files (Kconfig fragments)'.
- From 'Select device tree overlay...' to 'Base Devicetree overlays'.
- From 'Add code snippet...' to 'Snippets'.
- From 'Give a name for the current build directory' to 'Build directory name'.
- From 'Use Build system default or Use sysbuild' to 'System build (sysbuild)'.

Kconfig overlay added to base Kconfig

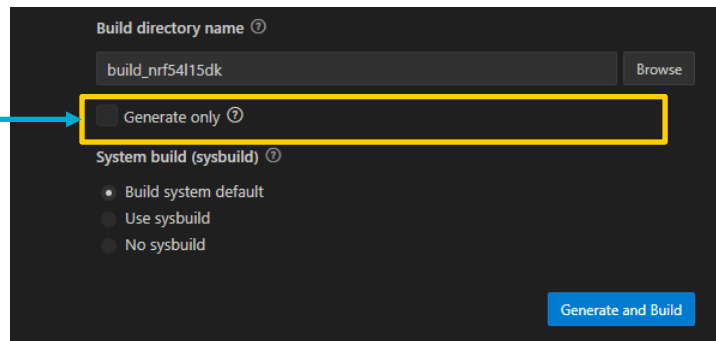
Additional devicetree overlay added to base devicetree

Select **Optimize for debugging**

Add Kconfig fragments and extra Cmake arguments if needed

Setting up build configurations 3/3

Keep box unchecked to generate the configuration and directly build the project.



The screenshot shows a dark-themed dialog box for build configuration. At the top, it says "Build directory name" with a help icon. Below this is a text input field containing "build_nrf54l15dk" and a "Browse" button. A yellow rectangular box highlights the "Generate only" checkbox, which is currently unchecked. A blue arrow points from the text "Keep box unchecked to generate the configuration and directly build the project." to this checkbox. Below the checkbox is the label "System build (sysbuild)" with a help icon. Underneath are three radio button options: "Build system default" (selected), "Use sysbuild", and "No sysbuild". At the bottom right of the dialog is a blue button labeled "Generate and Build". A blue arrow points from the text "Finally, hit Generate and Build" to this button.

Finally, hit Generate and Build

Tip: Press **CTRL+J** and select the “Terminal” tab page to view to progress of the building of the configuration

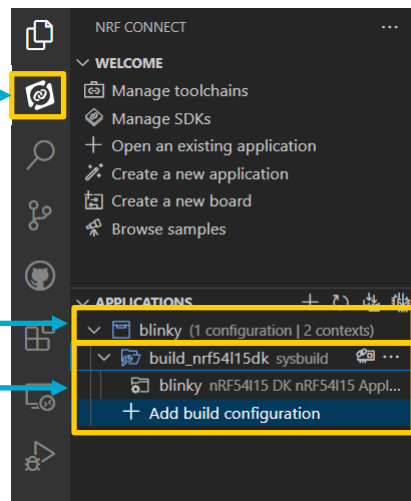
Build completed 1/2

Select the **nRF Connect** extension

Refers to the “**blinky**” project in the **VC Workspace**

Show the available **build configurations** for this application.

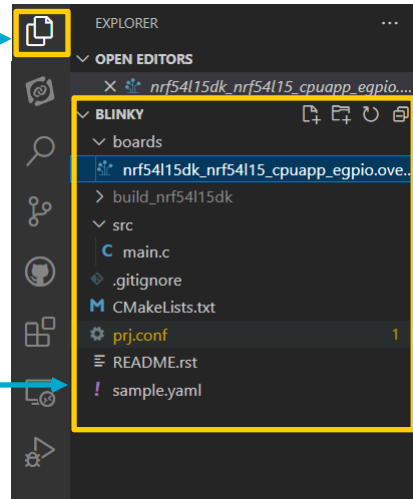
Multiple different builds are possible for a application. For example a debug and a production build.



Build completed 2/2

Select the **File explorer** extension

Show the available files in with in the application project.



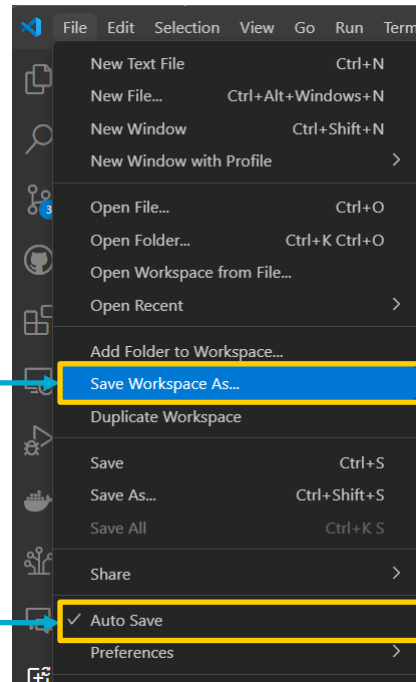
Visual Code – Workspace project

It is always advised to save your application into a Visual Code Workspace. The workspace contains configuration and setting specific of the project.

Don't confuse the VC Workspace with a Zephyr Workspace.

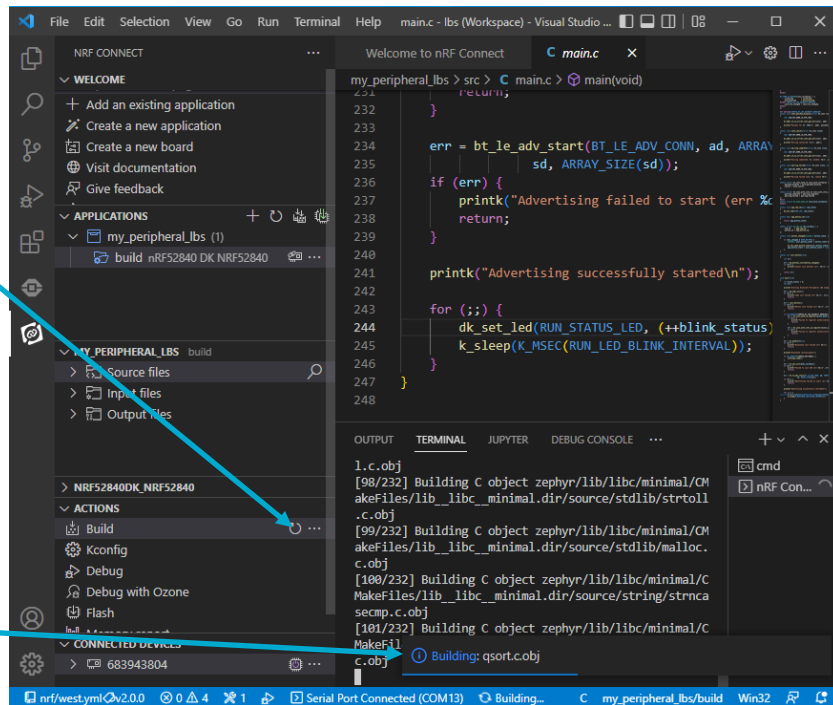
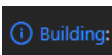
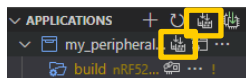
Save your Workspace

Enable Auto-Save



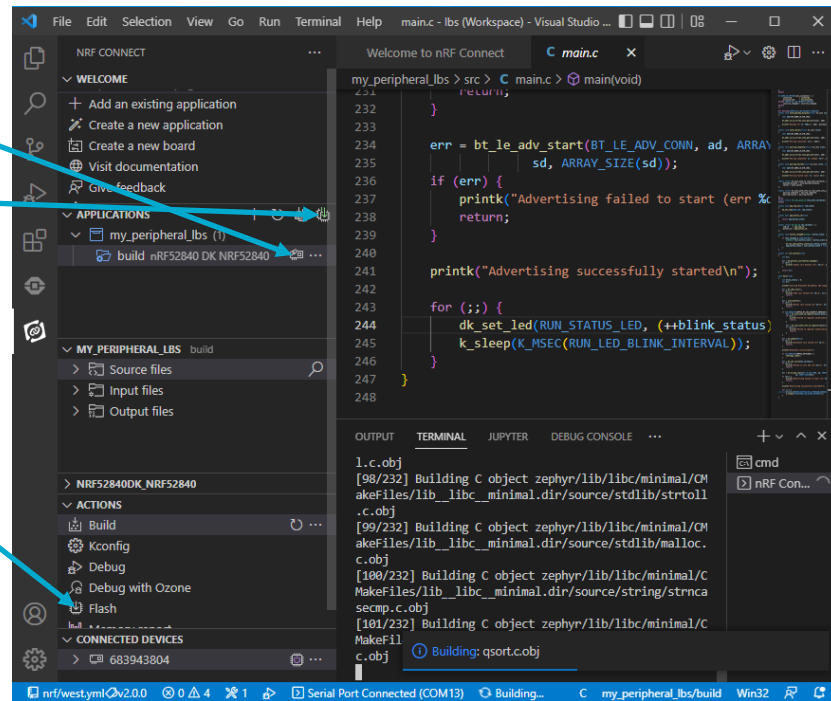
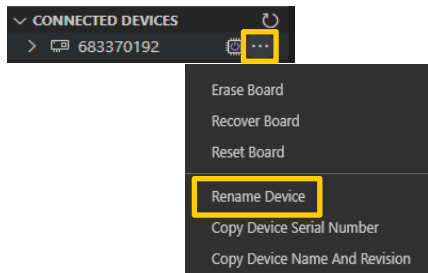
Building your first app

- To (re)build your app click “build” in the Actions view
 - Click here to force a pristine build (a forced clean build)
- If you have several applications or build configurations, you can build all with one click:
- Building progress can be viewed by clicking

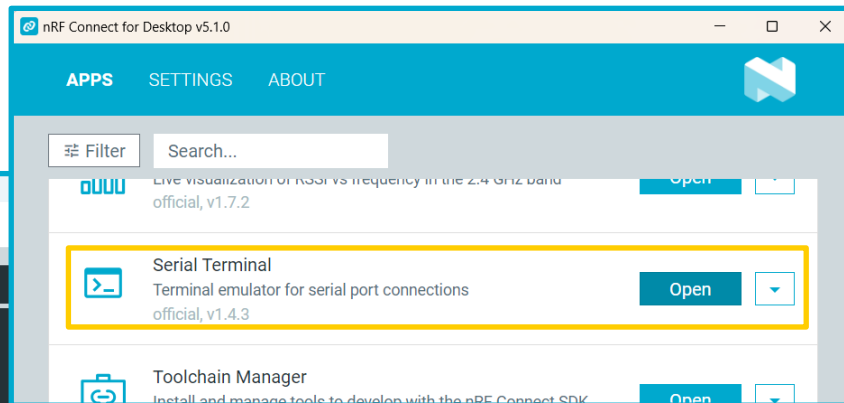
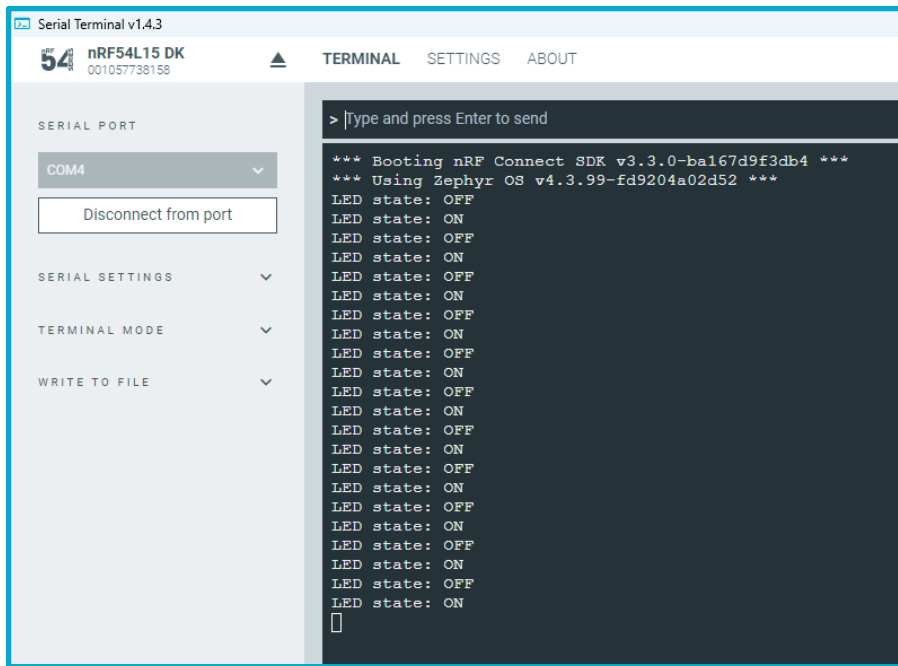


Flashing connected boards

- Link your build config to a connected board
- And flash all linked boards with one click
- Flash your active build
- Give your board a name:



nRF Connect For Desktop Serial Terminal



Playing with Blinky

<https://academy.nordicsemi.com/topic/exercise-2-1/>



Zephyr Sample: Blinky

In the following slides, we will examine the blinky sample line by line to understand how this program works.

The blinky sample comes as part of the nRF Connect SDK and is found at this location:

`<ncs>\zephyr\samples\basic\blinky`

```
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

/* 1000 msec = 1 sec */
#define SLEEP_TIME_MS 1000

/* The devicetree node identifier for the "led0" alias. */
#define LED0_NODE DT_ALIAS(led0)

static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(LED0_NODE, gpio);

int main(void)
{
    int ret;
    bool led_state = true;

    if (!gpio_is_ready_dt(&led)) {
        return 0;
    }

    ret = gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);
    if (ret < 0) {
        return 0;
    }

    while (1) {
        ret = gpio_pin_toggle_dt(&led);
        if (ret < 0) {
            return 0;
        }
        led_state = !led_state;
        printf("LED state: %s\n", led_state ? "ON" : "OFF");
        k_msleep(CONFIG_BLINKY_TIME_ON);
    }
    return 0;
}
```

Include Modules

The blinky sample uses the following modules of the nRF Connect SDK/Zephyr:

- C Standard library `<stdio.h>` for IO, printf
- Kernel services `<zephyr/kernel.h>` for the sleep function `k_msleep()` or kernel `printk()`
- The generic GPIO interface `<drivers/gpio.h>` for the structure `gpio_dt_spec`, the macros `GPIO_DT_SPEC_GET()`, and the functions `gpio_pin_configure_dt()` and `gpio_pin_toggle_dt()`.

The modules are included through the following include lines:

```
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>
```

Define the Node Identifier

- The line below uses the devicetree macro **DT_ALIAS()** to get the node identifier symbol **LED0_NODE**, which will represent node **led_0**.
- The **led_0** node is defined in the devicetree of the nRF54L15-DK. **LED0_NODE** is now a source code symbol that represents the hardware for **LED0**.
- The **DT_ALIAS()** macro gets the node identifier from the node's alias name, which as we saw in the Devicetree section, is **led0**.

```
/* The devicetree node identifier for the  
   "led0" alias. Defined in the .dts file */  
#define LED0_NODE DT_ALIAS(led0)
```

NOTE:

There are many ways to retrieve the node identifier. The macros **DT_PATH()**, **DT_NODELABEL()**, **DT_ALIAS()**, and **DT_INST()**. All return the node identifier, based on different input parameters.

Retrieve Device pointer, Pin nb and config flags

- The macro call `GPIO_DT_SPEC_GET()` returns the structure `gpio_dt_spec led`, which contains the device pointer for node `led_0` as well as the pin number and associated configuration flags.
- The node identifier `LED0_NODE` has this information embedded inside its `gpios` property.
- The second parameter `gpios`, the name of the property containing all this information.

```
static const struct gpio_dt_spec led =  
    GPIO_DT_SPEC_GET(LED0_NODE, gpios);
```

Verify that device is ready to use

Now we must pass the device pointer of the device, in this case the `led`, to `gpio_is_ready_dt()`, to verify that the device is ready for use.

```
int ret;
bool led_state = true;

if (!gpio_is_ready_dt(&led)) {
    return 0;
}
```

Configure GPIO Pin

The generic GPIO API function `gpio_pin_configure_dt()` is used to configure the GPIO pin

The led pin has an output (**active high**) and initializes it to a logic 1.

```
ret = gpio_pin_configure_dt(&led,  
                           GPIO_OUTPUT_ACTIVE);  
if (ret < 0) {  
    return;  
}
```

Continuously toggle the GPIO Pin

Finally, the blinky main function will enter an infinite loop where we continuously toggle the GPIO pin using `gpio_pin_toggle_dt()`.

In every iteration, we are calling the kernel service function `k_msleep()`, which puts the main function to sleep for 1 second

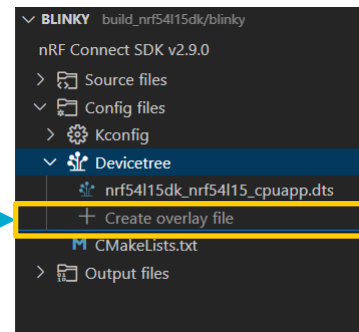
```
while (1) {  
    ret = gpio_pin_toggle_dt(&led);  
    if (ret < 0) {  
        return;  
    }  
    k_msleep(SLEEP_TIME_MS);  
}
```

Update our Blinky

Change the LED Pin

Change the LED using devicetree overlays

- First step: Create an overlay file
- When the file name is identical to the specified board in the “build configuration” it will be used automatically
- When using the nRF54L15DK the file name is **“nrf54l15dk_nrf54l15_cpuapp.overlay”**
- When using the EVK NORA B2 the file name is **“ubx_evknorab2_nrf54l15_cpuapp.overlay”**
- Use VC to create an overlay file
- TIP: Skip the pristine build & first perform the changes in the overlay file, then do a pristine build!



Option A: overwrite led0 node in overlay file

- Blinky uses the “led0” node referenced in main.c

```
14  /* The devicetree node identifier for the "led0" alias. */  
15  #define LED0_NODE DT_ALIAS(led0)
```

- Overwrite led0 to a different pin:

```
&led0 {  
    gpios = <&gpio2 7 GPIO_ACTIVE_HIGH>;  
};
```

nRF54L15-DK HW Setup:

LED0 = Port 2, Pin 9
LED1 = Port 1, Pin 10
LED2 = Port 2, Pin 7
LED3 = Port 1, Pin 14

EVK-NORA-B2 HW Setup:

(red) LED0 = Port 2, Pin 9
(green) LED1 = Port 2, Pin 7
(blue) LED2 = Port 1, Pin 10

Option B: Create a new DTS node group

- Create the DTS node group “**board_leds**” and add the node **my_led1** with its node label **my_led_1**

- ```
/ {
 board_leds {
 compatible = "gpio-leds";
 my_led_1: my_led1 {
 gpios = <&gpio1 10 (GPIO_ACTIVE_HIGH)>;
 label = "My custom LED";
 };
 };
};
```

- Change devicetree node identifier in **main.c** to new node definition (**my\_led1**).

```
/* The devicetree node identifier for the "led0" alias. */
#define LED0_NODE DT_PATH(board_leds, my_led1)
```

## Option C: Refer with alias 1/2

- Just as example we could also used `DT_ALIAS` instead of `DT_PATH`.
- Add an alias to the `board_leds` node

```
aliases {
 led0 = &my_led_1;
};
```

- This defines an alias to refer `led0` now to new defined `my_led1`
- Change in `main.c` the node identifier back to the alias `led0`.

```
/* The devicetree node identifier for the "led0" alias. */
#define LED0_NODE DT_ALIAS(led0)
```

## Option C: Refer with alias 2/2

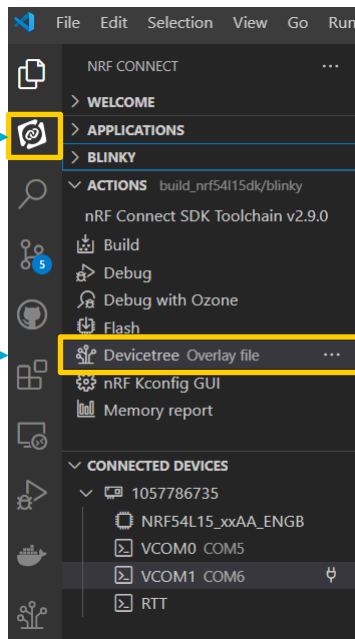
- Node definition with the alias added:

```
/ {
 board_leds {
 compatible = "gpio-leds";
 my_led_1: my_led1 {
 gpios = <&gpio1 10 (GPIO_ACTIVE_LOW)>;
 label = "My custom LED";
 };
 };
 aliases {
 led0 = &my_led_1;
 };
};
```

# Open visual devicetree editor

Select the nRF Connect  
Extension

In the actions section  
Select “Devicetree” to open  
the visual editor



Modifies directly the devicetree files.

Be careful that changes could  
effect not only this application.  
Some changes can be written into  
the board files.

The screenshot displays the Nordic Semiconductor DeviceTree Visual Editor interface. The left sidebar shows the project structure with the following components:

- DEVICETREE VISUAL EDITOR
- BUILD CONTEXTS
- CONTEXT FILES
- CONTEXT OVERVIEW
- NODES: blinky: build\_nrf54115dk/bl...
- aliases
- board\_leds
- buttons
- chosen
- clocks
- cpus
- leds
  - led0
  - led1
  - led2
  - led3
- pinctrl
- psa\_rng
- pwmleds
- rng\_hci
- soc

The main workspace shows the configuration for the **led0** node. The **PROPERTIES** section is expanded, showing:

- label: Green LED 0
- PINS: P2.09
- Assign...

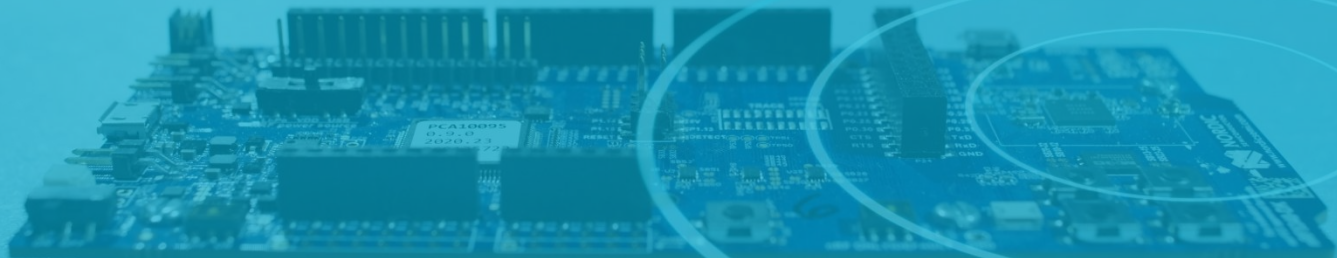
The **ADVANCED** section is collapsed, showing "No properties to show."

The **DESCRIPTON** section is expanded, showing "GPIO LED child node".

The right sidebar shows the configuration for the **leds** node. The **pins** section is expanded, showing:

- led0: P2.09
- led1: P1.10
- led2: P2.07
- led3: P1.14
- My Green LED 1: P1.14

The bottom status bar shows the project name "blinky" and the file "main.c 1, M".



**NORDIC**<sup>®</sup>  
SEMICONDUCTOR

# Update our Blinky

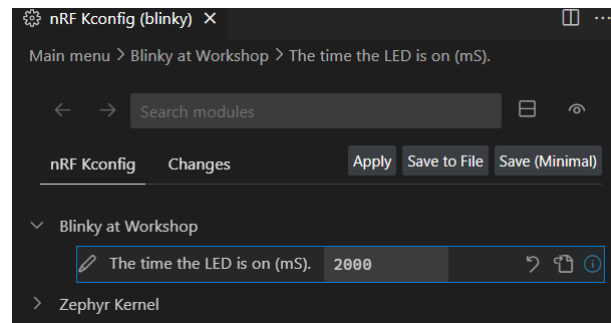
## Adding kernel definitions

# Build a kernel configuration menu

- Insert a new file in the root of the “**blinky**” project with the name “**Kconfig**”
- This defines **CONFIG\_BLINKY\_ON\_TIME** with a default value of “**1000**”
- Select the menu “nRF Kconfig GUI” in the “ACTIONS” section in the nRF Connect Ext.

```
menu "Blinky at Workshop"
config BLINKY_TIME_ON
 int "The time the LED is on (mS)."
 default 1000
endmenu

menu "Zephyr Kernel"
source "Kconfig.zephyr"
endmenu
```



# Add to application: edit manually

- Within our “**prj.conf**” we can modify the value **CONFIG\_BLINKY\_ON\_TIME**
- Add the following into the “**prj.conf**” file in your blinky project.

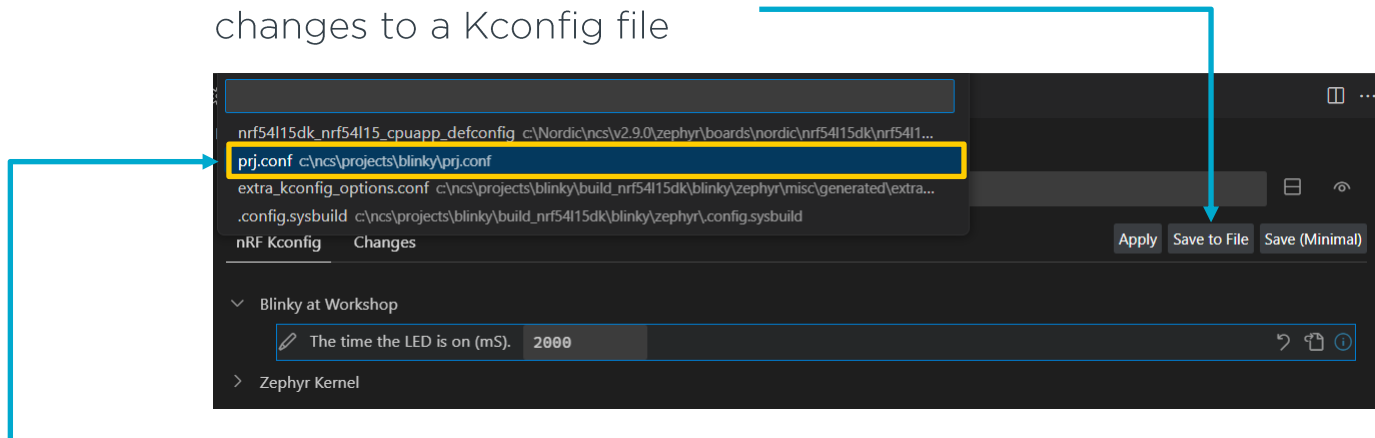
```
CONFIG_BLINKY_TIME_ON=2000
```

- Now replace in our **main.c** the wait (**while(1)**) loop with this definition:

```
k_msleep(CONFIG_BLINKY_TIME_ON);
```

# Add to application: nRF Kconfig GUI

Select Save to file to make the changes to a Kconfig file



Select Kconfig file to make modify the changed values.  
Use the prj.conf file of the current application.

Selecting the nrf54l15dk\_nrf54l15\_cpuapp\_defconfig will modify the SDK for all applications.  
When written in extra\_kconfig\_options.conf the modification is removed with pristine build

Update our Blinky  
Make use of Zephyr work queues

# Modify blinky using system work queue

- Instead of blinking the **LED** in the main loop we move it to a system thread.
- First step is to create a delayable work queue

```
static struct k_work_delayable blink_led_work;
```

- Create a **work\_queue** handler function

```
static void blink_led_work_fn(struct k_work *work)
{
 int ret;
 ret = gpio_pin_toggle_dt(&led);
 if (ret < 0) {
 return;
 }
 k_work_schedule(&blink_led_work, K_MSEC(CONFIG_BLINKY_TIME_ON));
}
```

# Modify blinky using system work queue

- Create function for initializing the work queues

```
static void work_init(void)
{
 k_work_init_delayable(&blink_led_work, blink_led_work_fn);
}
```

- Call the init routine in `main.c` remove `while(1)` loop and add schedule the delayable work queue immediately with option `K_NO_WAIT`

```
work_init();

k_work_schedule(&blink_led_work, K_NO_WAIT);
```